| | Array | Collection |
|---|---|---|
| 长度 | 固定[0, Integer.MAX_VALUE - 4] | 可变（自动扩容） |
| 元素类型 | 同一类型（包括基础类型） | 多种引用类型（不包括基础类型） |
| 底层实现 | > 基础数据类型：JVM启动时在方法区给每个基础数据类型创建TypeArrayKlass实例<br>创建数组实际上是在heap上创建了typeArrayOop实例<br>> Object：加载类时在方法区创建InstanceKlass实例<br>创建数据实际上是在heap上创建objArrayOop实例 | 数组 or 链表 |
| 其他 | | > 类型丰富：List、Map、Set、Queue<br>> 功能丰富：支持并发、迭代、计数、自动扩容等 |

| 容器类型 | 并发类型 | 队列名称 | 容量 | 线程同步方式 | 数据结构 | 使用场景 | 其他 |
|---|---|---|---|---|---|---|---|
| queue | 非阻塞 | ConcurrentLinkedQueue | 无界 | cas + 自旋 | 单链表 | 并发环境，需要栈or队列数据结构时 | |
| queue | 阻塞 | LinkedBlockingQueue | 有界：caller指定容量，默认值Integer.MAX_VALUE | ReentrantLock（读写分离） | 单链表 | FixedThreadPool | 阻塞队列，一般用于生产者-消费者模式<br>用于线程之间交换数据or系统解耦 |
| queue | 阻塞 | ArrayBlockingQueue | 有界：caller指定容量 | ReentrantLock（fair/unfair） | 数组（环形队列） | | |
| queue | 阻塞 | PriorityBlockingQueue | 近似无界：最大容量Integer.MAX_VALUE - 8 | ReentrantLock | 数组（二叉树堆） | | |
| queue | 阻塞 | DlayedWorkQueue | 近似无界：最大值Integer.MAX_VALUE | ReentrantLock | 数组（二叉树堆） | ScheduledThreadPool | |
| queue | 阻塞 | DelayQueue | 近似无界：最大容量Integer.MAX_VALUE | ReentrantLock | 数组（二叉树堆） | 缓存、定时任务、异步通知失败重试 | |
| queue | 阻塞 | LinkedTransferQueue | 无界 | - | 单链表 | `pruducers can await receipt of elements, or enqueue elements directly` | |
| queue | 阻塞 | SynchronousQueue | 无界 | - | 单链表（stack / queue） | 线程阻塞，等待其他线程来交换数据，例如：CachedThreadPool | |

| | HashTable | HashMap | ConcurrentHashMap JDK1.8 | 备注 |
|---|---|---|---|---|
| 存储结构 | Entry数组：容量[11,Integer.MAX_VALUE - 8] | Node数组：容量[16, 2^30]<br>链 -> 树：链表长度超过8且capacity超过64<br>链 <- 树：树节点小于6 | Node数组：容量[16, 2^30]<br>链 -> 树：链表长度超过8且capacity超过64<br>链 <- 树：树节点小于6 | > 数组容量为什么是2的幂次？bucket坐标可以直接位运算，提高哈希寻址效率<br>> 哈希寻址？计算哈希值：key.hashCode高16位右移到低位，再和key.hashCode做异或运算得到hash（充分利用高16位的差异性）<br>计算bucket坐标：hash & (n-1)，n为当前数组长度，位运算速度较快，相当于截取hash的后(n-1)位，提高寻址速度<br>> 为什么要转树？如果用户自己实现hashCode离散程度不好，链表会变得很长，降低查询效率，红黑树在节点较多的时候检索性能比链表高<br>> 为什么不直接用树？TreeNode是Node的2倍大，hash算法正常的话，链表不会很长，红黑树不会带来明显的查询效率优势，反而增加内存负担<br>> 转树的阈值为什么是8？hashCode离散程度较好的话，会均匀分布，很少出现链表很长的情况，链表长度为8时的概率小于千万分之一 |
| key-value限制 | Key和value都不能为空 | N/A | Key和value都不能为空：value允许为null时会存在二义性（分不清是key不存在还是value为null） | |
| put | | | | |
| get | | | get操作的线程安全性：单线程写，多线程并发读<br>> treeify or untreeify：不破坏链表结构，可以读取<br>> fwd node：迁移了，到nextTable中读取<br>> reserve node：正在计算，返回null<br>> 链表put：新Node都是加入链表尾部，不影响读取<br>> 链表remove：被remove的Node next引用还在，链表没断，能正常读<br>> 链表replace：链表结构不变，只是value有更新<br>> 红黑树put/remove/replace：内置轻量级读写锁，读写操作互斥（允许多个线程同时读取，但不能和写操作并行） | |
| remove | | | | |
| count | int count计数 | int size计数 | > 无并发冲突时：使用long baseCount计数<br>> 并发冲突时：分散热点，线程随机选择一个CounterCell计数，如果CAS cell冲突，重新选择一个，必要时扩容<br>> 总数 = baseCount + sum(counter cells) | 为什么不用volatile+cas之类的线程同步技术？map count更新频繁，会导致cpu缓存频繁刷新，影响性能 |
| Iterator | fail-fast强一致性迭代：同HashMap | > fail-fast强一致性迭代：遍历前记录map此时的modCount，每次获取next Node时先检查，如果modCount变了，抛异常<br>> foreach Map.entrySet()遍历键值对，内置HashIterator遍历所有buckets | > fail-safe弱一致性迭代：遍历过程中，允许其他线程同时修改数据，如果修改的是已经遍历过的数据<br>> 遇到fwd节点时，说明bucket已迁移至nextTable > 将当前table的遍历状态保存到stack，切换至nextTable<br>> 遍历nextTable对应2个bucket数据（bucket扩容后会变2个）<br>> 根据stack中的记录恢复数据，继续遍历当前table | |
| 线程同步 | synchronized方法，锁整个map | N/A | 1.table的可见性：volatile修饰，保证table在多线程中的一致性<br>2.bucket的可见性：> tabAt()时：使用Unsafe.getObjectVolatile()保证多线程间的可见性<br>> bucket为null时：使用Unsafe的CAS方法更新<br>> bucket不为null时：1) 通过synchronized block获取table[I]处Node对象锁<br>2) 拿到锁后，立即双重校验，确保Node没变化再执行操作<br>3.value的可见性：Node.value用volatile修饰 | |
| 扩容 | > 新建数组，容量扩一倍，替换原数组<br>> 迁移数据：遍历原数组所有buckets，迁移数据 | > 新建数组，容量扩一倍，替换原数组（并发问题，如果此时其他线程访问table，会找不到数据）<br>> 迁移数据：遍历原数组所有buckets，迁移数据 | > 新建数组，容量扩一倍，保存到map.nextTab<br>> 多线程迁移数据：线程之间通过transferIndex同步迁移进度，不会有冲突<br>每个线程负责一定数量的buckets，迁移完的bucket放置特殊Node标识<br>> 最后完成迁移的线程检查所有buckets，如果都完成迁移，table指向nextTab，更新threhold | |

| | ... | 32 | 16 | 8 | 4 | 2 | 1 | bucket坐标 |
|---|---|---|---|---|---|---|---|---|
| 数组长度=16 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 15 & hash值相当于取hash值的后4位：8X3+4X2+2X1+X0 |
| 数组长度=32 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 扩容后，31 & hash值相当于取hash值的后5位<br>> X4为0的Node坐标：0+8X3+4X2+2X1+X0（和扩容前相同）<br>> X4为1的Node坐标：16+8X3+4X2+2X1+X0（16+原坐标） |
| key的hash值 | X6 | X5 | X4 | X3 | X2 | X1 | X0 | |